

DEVOPS IMPLEMENTATION & INTEGRATION PLAN

Automated CI/CD Pipeline, Containerization, and Cloud Deployment Project

Author: Adem Abassi	Methodology: Agile Scrum Framework	Scope: CI/CD, Docker, Render
Role: DevOps Intern	Sprint Duration: 1-Week Iterations (4 Weeks Total)	Registry: Docker Hub

1. PROJECT OVERVIEW & EXECUTIVE SUMMARY

This project plan defines the roadmap for establishing an automated continuous integration and continuous deployment (CI/CD) delivery mechanism for an existing software application hosted on GitHub. The goal is to transition the application lifecycle from manual deployment methods to a standardized DevOps workflow using containerization, automated testing, container registry distribution, and automated cloud hosting on Render.

- Key Project Objectives:**
- **Standardization:** Eliminate local environment inconsistencies by containerizing frontend and backend components via Docker and Docker Compose.
 - **Automation:** Implement GitHub Actions workflows to automate code verification, testing, and container image building upon repository updates.
 - **Deployment:** Establish continuous deployment directly to the Render cloud platform with built-in SSL and routing management.

2. CURRENT STATE ANALYSIS (AUDIT & GAP ASSESSMENT)

A comprehensive technical audit of the current application state was conducted to identify operational bottlenecks, deployment risks, and environment discrepancies.

Evaluation Domain	Current State (As-Is)	Identified Risk / Bottleneck
Build & Runtime	Dependencies installed directly on local or virtual machine operating system without isolation.	High probability of runtime failures due to "works on my machine" conflicts across environments.
Deployment Process	Manual execution of shell scripts and git pull commands directly on target host servers.	Human-error prone, inconsistent release procedures, and elevated application downtime during releases.
Verification & Testing	Manual test execution by developers prior to pushing code to remote branches.	Unverified code merged into production branches, leading to bug propagation and delayed feedback loops.

3. SYSTEM REQUIREMENT SPECIFICATIONS

3.1 Functional Requirements

- **FR-01 Containerization:** The system must package all application layers into standard Docker images using environment-agnostic Dockerfiles and orchestrate them using Docker Compose.
- **FR-02 Automated Integration (CI):** The pipeline must automatically execute code linting, unit tests, and Docker image builds upon every commit or Pull Request sent to the main branch.
- **FR-03 Registry Integration:** Verified Docker images must be automatically tagged and published to Docker Hub upon successful build validation.

- **FR-04 Automated Deployment (CD):** Pushing passing code updates to the target branch must trigger an automated deployment pipeline directly to the Render platform.

3.2 Non-Functional Requirements

- **NFR-01 Security & Secrets Management:** No production passwords, API tokens, or credentials shall be stored in source code; all secrets must be managed securely via GitHub Secrets and Render environment variables.
- **NFR-02 Maintainability & Simplicity:** All pipeline configurations (YAML files, Dockerfiles) must follow modular, well-documented structures to ensure easy maintenance.
- **NFR-03 Cost Efficiency:** The implementation must leverage zero-cost tier solutions (Docker Hub, GitHub Actions free allotment, Render free tier).
- **NFR-04 Reliability & Recovery:** Deployment procedures must ensure rapid recovery through version-tagged Docker image rollbacks if a release fails.

4. SPRINT-BASED IMPLEMENTATION PLAN (AGILE SCRUM)

The implementation is organized according to the Agile Scrum framework across four 1-week Sprints. Each sprint represents an increment delivering functional DevOps capabilities.

SPRINT 1 System Requirements Specifications, Architecture & Local Containerization

Goal: Finalize functional scope specifications and achieve full application containerization locally.

- Formulate system requirement specifications document and establish project backlog.
- Audit source repository structure, application dependencies, and configuration parameters.
- Write optimized multi-stage `Dockerfile` manifests for application components.
- Configure `docker-compose.yml` for unified local stack orchestration and testing.

Deliverables: System Requirements Document, Dockerfiles, docker-compose.yml

SPRINT 2 Continuous Integration (CI) Pipeline & Registry Automation

Goal: Build an automated GitHub Actions pipeline for testing and publishing Docker images.

- Configure Docker Hub repository access tokens and store encrypted credentials in GitHub Secrets.
- Develop GitHub Actions workflow (`.github/workflows/ci.yml`) triggered on code push/pull requests.
- Incorporate automated testing, code validation, and container image compilation steps.
- Automate image tagging (`latest` and commit SHA) and automated push to Docker Hub.

Deliverables: Verified CI Workflow, Automated Docker Hub Image Builds

SPRINT 3 Cloud Infrastructure Setup & Initial Deployment

Goal: Initialize Render cloud hosting environment and execute manual verification deployment.

- Set up Render cloud Web Service instance and configure environment variable definitions.
- Connect Render to Docker Hub to access built application images.
- Deploy the containerized application stack to Render to verify runtime compatibility.
- Validate built-in Render SSL provisioning and public web domain accessibility.

Deliverables: Render Cloud Instance, Verified Manual Deployment

SPRINT 4 Continuous Deployment (CD) Automation & Project Documentation

Goal: Automate continuous deployment triggers on Render and compile final project documentation.

- Configure deployment webhooks/triggers linking Docker Hub image updates or GitHub merges directly to Render.
- Perform end-to-end integration testing: code change -> GitHub Actions -> Docker Hub -> Render deployment.
- Document deployment workflow, operational runbooks, and repository maintenance instructions.
- Prepare final internship report presentation and hand over repository assets.

Deliverables: End-to-End Automated CI/CD Pipeline, Final Technical Report

5. TECHNOLOGICAL STACK SELECTION & RATIONALE

Technology Domain	Selected Tool	Rationale & Advantage for Selected Scope
Version Control	GitHub	Industry standard VCS with native GitHub Actions workflow integration and repository access controls.
Containerization	Docker / Compose	Provides complete environment isolation, lightweight resource consumption, and rapid reproducibility.
CI/CD Orchestration	GitHub Actions	Native repository automation without separate server management; free runner usage for standard repositories.
Container Registry	Docker Hub	Direct compatibility with Docker CLI, simple authentication management, and universal platform support.
Hosting Platform	Render	Native container runtime support, built-in SSL/routing management, automated push-to-deploy capabilities.

6. RISK MANAGEMENT & MITIGATION MATRIX

Risk Description	Severity	Mitigation Strategy
Free Tier Build Timeout	Medium	Optimize Dockerfile build cache utilizing multi-stage builds and layer caching mechanisms.
Secrets Leakage in Source	High	Enforce `.gitignore` policy, utilize environment variables, and store secrets in GitHub Secrets exclusively.
Deployment Failure on Host	High	Maintain previous stable image tags on Docker Hub to allow immediate rollback on Render if needed.